1

Unit 1: Number Systems & Codes · Boolean Algebra & Logic Gates

Digital Logic Design

CHANDRAS EDU

Never stop learning

Outline

1.1 Number Systems: Binary, Octal, Decimal, Hexadecimal
1.2 Fixed-Point & Floating-Point Representation
1.3 Complements of Numbers
1.4 Error Detection & Correction: Parity, Hamming Code
1.5 Boolean Algebra: Axioms, Theorems & Properties
1.6 Boolean Functions: Canonical & Standard Forms
1.7 Digital Logic Gates Overview

1.1 Number Systems: Binary, Octal, Decimal, Hexadecimal

IN SIMPLE TERMS

Every number system multiplies each digit by the base raised to its position; binary, octal and hex are just different bases, and octal and hex are shorthand for groups of 3 or 4 bits.

Positional number systems

- All four systems are **positional (radix) systems**: a digit's value depends on the digit and on its position
- A system with radix (base) r uses r digit symbols, 0 through r − 1
- A number is a weighted sum of powers of r: $\mathbf{N = \Sigma\ d_i \times r^i}$
- d_i is the digit at position i, counting from 0 at the radix point going left
- This one formula covers all four systems: only r changes

The four number systems

System	Radix	Digit set
Decimal	10	0–9
Binary	2	0, 1
Octal	8	0–7
Hexadecimal	16	0–9, A–F (A = 10 … F = 15)

Why octal and hexadecimal?

- Octal and hexadecimal exist purely for **human convenience**
- No circuit computes in base 8 or base 16 internally
- 3 binary digits map exactly onto 1 octal digit
- 4 binary digits map exactly onto 1 hex digit
- Long bit strings become far easier to read and write without losing the binary structure

Converting any base to decimal

$$1011_{2} = 1\times2^{3} + 0\times2^{2} + 1\times2^{1} + 1\times2^{0} = 8 + 0 + 2 + 1 = 11_{10}$$

$$2AF_{16} = 2\times16^{2} + A\times16^{1} + F\times16^{0}$$

$$= 2\times256 + 10\times16 + 15\times1 = 512 + 160 + 15 = 687_{10}$$

- Directly apply the weighted-sum formula, evaluating in decimal

Converting decimal to any base

- Use **repeated division by the target radix**
- Read the remainders from **bottom to top**
- The last remainder computed is the most significant digit
- The next slide converts decimal 45 to binary and to octal

Worked example: decimal 45 to binary and octal


```
45 ÷ 2 = 22 remainder 1   (LSB)
22 ÷ 2 = 11 remainder 0
11 ÷ 2 = 5 remainder 1
5 ÷ 2 = 2 remainder 1
2 ÷ 2 = 1 remainder 0
1 ÷ 2 = 0 remainder 1   (MSB)
Reading remainders bottom-to-top: 45₁₀ = 101101₂
45 ÷ 8 = 5 remainder 5   (LSB)
5 ÷ 8 = 0 remainder 5   (MSB)   →   45₁₀ = 55₈
```

- Check: $101101_2 = 32 + 8 + 4 + 1 = 45$ ✓, and $55_8 = 5\times8 + 5 = 45$ ✓

Direct grouping: binary ↔ octal and hex

- Because $8 = 2^3$ and $16 = 2^4$, no decimal step is needed
- **Binary → octal:** group bits in **3s** from the radix point outward, pad with zeros if needed, convert each group
- **Binary → hexadecimal:** group bits in **4s** from the radix point outward, same padding rule
- **Octal/hex → binary:** expand each digit back into its fixed-width bit group and concatenate

Worked example: binary to octal and hex by grouping

$10\ 110\ 101_2$ (grouped in 3s from the right)

$2\quad 6\quad 5 = 265_8$

$1011\ 0101_2$ (grouped in 4s from the right)

$B\quad 5 = B5_{16}$

Conversion summary

Conversion	Method
Any base → decimal	Weighted sum: $\Sigma\, d_i \times r^i$
Decimal → any base	Repeated division by the target radix; read remainders bottom-to-top
Binary ↔ octal	Group bits in 3s from the radix point
Binary ↔ hexadecimal	Group bits in 4s from the radix point

1.2 Fixed-Point & Floating-Point Representation

IN SIMPLE TERMS

Fixed point keeps the binary point in one place; floating point stores the digits and, separately, where the point goes, like scientific notation.

Fixed-point representation

- A **fixed-point** number puts the binary point at a permanently agreed position within a fixed-width word
- Some bits hold the integer part and the rest the fractional part, decided once at design time
- Bits to the right of the point carry **negative** powers of 2: $b_{-1}b_{-2}b_{-3}\ldots$
- Each fractional bit b_{-i} contributes $b_{-i} \times 2^{-i}$
- **Trade-off:** more fractional bits mean finer precision but fewer integer bits and a smaller maximum magnitude
- Range and precision are **locked in** at design time

Worked example: a fixed-point binary fraction

$$1011.101_2 = (1\times2^2 + 0\times2^1 + 1\times2^0) + (1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3})$$
$$= (4 + 0 + 1) + (0.5 + 0 + 0.125)$$
$$= 5 + 0.625 = 5.625_{10}$$

Floating-point representation

- A **floating-point** number stores a sign, a **mantissa** (significand) and an **exponent**, like scientific notation
- The exponent says how far, and in which direction, to shift the binary point: the point “floats”
- **Value = $(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{\text{exponent} - \text{bias}}$**
- A number is **normalized** when exactly one nonzero digit sits before the binary point
- In binary that leading digit is always 1, so it need not be stored: the **implied leading 1** (hidden bit) gives one extra bit of precision for free

IEEE 754 single precision (32-bit)

Field	Width	Purpose
Sign	1 bit	0 = positive, 1 = negative
Exponent	8 bits	Stored with a bias of 127, so it can represent both positive and negative exponents using only unsigned bits
Mantissa (fraction)	23 bits	The bits after the implied leading 1

Worked example: encoding 5.625 in IEEE 754 single precision

$5.625_{10} = 101.101_2$ (from the fixed-point example above)

Normalize: $101.101_2 = 1.01101 \times 2^2$

Sign = 0 (positive)

Exponent = 2 + 127 = 129 = 10000001_2

Mantissa = 01101000000000000000000 (23 bits, padded with 0s)

Final layout: 0 10000001 01101000000000000000000

sign exponent(8) mantissa(23)

Fixed-point versus floating-point

	Fixed-point	Floating-point
Range	Fixed, limited by word width	Very wide: the exponent lets it represent tiny and huge magnitudes
Precision	Constant across the whole range	Relative: more precise near zero, coarser for huge magnitudes
Hardware cost	Cheap: plain integer adders/multipliers work directly	Expensive: needs dedicated floating-point units (FPUs) for normalization and exponent alignment
Typical use	Embedded/DSP systems with known, bounded value ranges	General-purpose computing, scientific calculation

1.3 Complements of Numbers

IN SIMPLE TERMS

Flip every bit for the 1’s complement, then add 1 for the 2’s complement; the computer uses it to subtract by adding.

Two kinds of complement

- For radix r there are two standard complements of an n-digit number N
- The **(r−1)'s complement:** subtract each digit of N from (r−1)
- The **r's complement:** the (r−1)'s complement plus 1 added to the least significant digit
- In binary (r = 2) these are the **1's complement** and the **2's complement**
- Negative numbers are represented so that "subtract B" becomes "add the complement of B"

1’s complement

- The 1’s complement is found by **flipping every bit**: 0 becomes 1, 1 becomes 0
- This is “subtract each digit from 1”, phrased as bitwise NOT
- It has **two representations of zero**: 00000000 (positive zero) and 11111111 (negative zero)
- This complicates hardware comparison and arithmetic logic, which 2’s complement fixes

Example: 1’s complement

N = $1010\ 0110_2$

1’s comp = $0101\ 1001_2$ (every bit flipped)

2’s complement

- **2’s complement = (1’s complement) + 1**
- It has exactly **one** representation of zero: the +1 makes the complement of 11111111 wrap around to 00000000
- This is why essentially every real computer uses 2’s complement for signed integers
- **Shortcut:** scan from the least significant bit, copy every bit up to and including the first 1, then flip every bit after that
- Both methods always agree; the shortcut is faster by hand

Example: 2’s complement

N = $1010\ 0110_2$

1’s comp = $0101\ 1001_2$

+1 = 0000 0001

2’s comp = $0101\ 1010_2$

Subtraction via complement addition

- To compute A − B with n-bit r’s complement arithmetic: **add A to the r’s complement of B**
- Then apply the **end-carry rule**:
- **Carry out of the MSB:** discard it; the remaining n bits are the correct, positive result
- **No carry:** the result is negative and is itself in r’s complement form; take its r’s complement to read the magnitude

Worked example: 9 – 6 using 2’s complement (4-bit)


```
A = 9 = 1001₂
B = 6 = 0110₂
2’s complement of B = 1010₂
   1001
+  1010
  ------
1 0011   ← carry out of MSB: discard it
```

- Result: 0011_2 = 3. Check: 9 – 6 = 3 ✓

Worked example: 6 – 9 using 2’s complement (4-bit, negative result)


```
A = 6 = 0110₂
B = 9 = 1001₂
2’s complement of B = 0111₂
   0110
+  0111
  ------
   1101   ← no carry out of MSB: result is negative
```

- 2’s complement of $1101_2 = 0011_2 = 3$. Result: −3. Check: 6 − 9 = −3 ✓

1.4 Error Detection & Correction: Parity, Hamming Code

IN SIMPLE TERMS

A parity bit tells you an error happened; a Hamming code adds a few more parity bits so it can also tell you which bit to fix.

Parity check

- A **parity bit** is one extra bit appended to a group of data bits
- It makes the total number of 1-bits in the whole group (data + parity) match an agreed convention
- **Even parity:** the total count of 1s is even
- **Odd parity:** the total count of 1s is odd
- The even parity bit is the XOR of all the data bits
- The receiver recomputes parity: a mismatch means **at least one bit was corrupted**, but not which one

Worked example: even parity


```
Data: 1011000   (four 1s: already even)
Even parity bit = 0
Transmitted: 1011000 0

Data: 1011001   (five 1s: odd)
Even parity bit = 1
Transmitted: 1011001 1
```

Hamming code

- Parity tells you that an error happened
- **Hamming code** uses **multiple parity bits**, each covering a carefully chosen overlapping subset of positions
- This pinpoints exactly which bit is wrong
- Knowing the location of a single-bit error is enough to fix it: just flip that bit
- So Hamming code can **self-correct** single-bit errors without retransmission

Placing the parity bits

- Number the bit positions of the codeword starting at 1
- Parity bits go at every position that is a **power of 2** (1, 2, 4, 8, …); data bits fill every other position
- For 4 data bits: 3 parity bits (positions 1, 2, 4) and a 7-bit codeword: **Hamming(7,4)**
- $\mathbf{p_1}$ covers every position with bit 0 set: 1, 3, 5, 7
- $\mathbf{p_2}$ covers every position with bit 1 set: 2, 3, 6, 7
- $\mathbf{p_4}$ covers every position with bit 2 set: 4, 5, 6, 7

Hamming(7,4) layout

Position	1	2	3	4	5	6	7
Role	p1	p2	d1	p4	d2	d3	d4

Worked example: encoding data 1011


```
Positions:   1  2  3  4  5  6  7
Role:        p1 p2 d1 p4 d2 d3 d4
Data placed:  ?  ?  1  ?  0  1  1
p1 (covers 1,3,5,7) = d1⊕d2⊕d4 = 1⊕0⊕1 = 0
p2 (covers 2,3,6,7) = d1⊕d3⊕d4 = 1⊕1⊕1 = 1
p4 (covers 4,5,6,7) = d2⊕d3⊕d4 = 0⊕1⊕1 = 0
Final codeword:  0  1  1  0  0  1  1
```

Worked example: detecting and correcting a single-bit error


```
Sent: 0 1 1 0 0 1 1     Received (position 5 flipped): 0 1 1 0 1 1 1
c1 = XOR of positions 1,3,5,7 = 0⊕1⊕1⊕1 = 1   (mismatch)
c2 = XOR of positions 2,3,6,7 = 1⊕1⊕1⊕1 = 0   (match)
c4 = XOR of positions 4,5,6,7 = 0⊕1⊕1⊕1 = 1   (mismatch)
Syndrome = c4 c2 c1 = 1 0 1 = binary 101 = 5
→ Position 5 is the error. Flip it back: 1 → 0
→ Corrected codeword: 0 1 1 0 0 1 1   (matches the original)
```

- The syndrome, read as a binary number, gives the position of the flipped bit; all 0s means no error

1.5 Boolean Algebra: Axioms, Theorems & Properties

IN SIMPLE TERMS

Boolean algebra is ordinary algebra for 0 and 1; a short list of rules lets you simplify any logic expression.

Huntington’s postulates

- A Boolean algebra is a set of elements with two binary operators (+ for OR, · for AND) and one unary operator (’ for NOT)
- It must satisfy a list of postulates (E. V. Huntington, 1904)
- The distributive law’s OR form, **x + (y·z) = (x+y)·(x+z)**, has no equivalent in ordinary algebra
- It is a defining feature of Boolean algebra

The postulates

Postulate	OR form	AND form
Closure	x + y is in the set	x · y is in the set
Identity element	x + 0 = x	x · 1 = x
Commutative	x + y = y + x	x · y = y · x
Distributive	x + (y·z) = (x+y)·(x+z)	x·(y+z) = x·y + x·z
Complement	x + x′ = 1	x · x′ = 0

The duality principle

- Every postulate comes in a matched pair
- In any true Boolean statement, swap every **+** with ·, every · with **+**, every **0** with **1** and every **1** with **0**
- The result, its **dual**, is also guaranteed true
- This is not a coincidence to memorise: it follows from the symmetric pairs of postulates

Basic theorems

Theorems are derived from the postulates, but are used just as heavily for simplification.

Theorem	OR form	AND form
Idempotent	$x + x = x$	$x \cdot x = x$
Null (annulment)	$x + 1 = 1$	$x \cdot 0 = 0$
Involution	$(x')' = x$	
Associative	$x + (y+z) = (x+y) + z$	$x \cdot (y \cdot z) = (x \cdot y) \cdot z$
Absorption	$x + xy = x$	$x(x+y) = x$
DeMorgan's	$(x+y)' = x'y'$	$(xy)' = x'+y'$

DeMorgan's laws in plain language

- "NOT (A OR B)" is the same as "(NOT A) AND (NOT B)"
- "NOT (A AND B)" is the same as "(NOT A) OR (NOT B)"
- A NOR gate can be redrawn as an AND gate with both inputs inverted
- A NAND gate can be redrawn as an OR gate with both inputs inverted
- You will use this constantly once gate-level implementation comes up

Proving the absorption theorem: x + xy = x


```
x + xy = x·1 + x·y      (identity: x = x·1)
       = x·(1 + y)      (distributive)
       = x·1            (null theorem: 1 + y = 1)
       = x              (identity)
```

Worked example: simplifying F = AB + AB’ + A’B


```
F = AB + AB' + A'B
  = A(B + B') + A'B        (distributive, factoring A out of the first
two terms)
  = A·1 + A'B              (complement: B + B' = 1)
  = A + A'B                (identity)
  = A + B                  (absorption-style identity: x + x'y = x + y)
```

- Three terms and multiple gates collapse into a simple two-variable OR

1.6 Boolean Functions: Canonical & Standard Forms

IN SIMPLE TERMS

A truth table can be written as a sum of the rows where F = 1 (minterms) or a product of the rows where F = 0 (maxterms).

Minterms and maxterms

- For n variables, a **minterm** is an AND term that includes every variable exactly once: uncomplemented if the variable is 1 in that row, complemented if 0
- A **maxterm** is the dual: an OR term including every variable, uncomplemented if 0 in that row, complemented if 1
- $\mathbf{m_i = M_i'}$ for every row: a minterm and its same-numbered maxterm are complements
- This is what DeMorgan's law produces when an AND of literals is complemented

Minterms and maxterms of three variables

A	B	C	Minterm	Maxterm
0	0	0	$m_0 = A'B'C'$	$M_0 = A+B+C$
0	0	1	$m_1 = A'B'C$	$M_1 = A+B+C'$
0	1	0	$m_2 = A'BC'$	$M_2 = A+B'+C$
0	1	1	$m_3 = A'BC$	$M_3 = A+B'+C'$
1	0	0	$m_4 = AB'C'$	$M_4 = A'+B+C$
1	0	1	$m_5 = AB'C$	$M_5 = A'+B+C'$
1	1	0	$m_6 = ABC'$	$M_6 = A'+B'+C$
1	1	1	$m_7 = ABC$	$M_7 = A'+B'+C'$

Canonical sum-of-products (SOP)


```
F(A,B,C) is 1 for rows 1, 4 and 7
(A'B'C, AB'C', ABC)
F = A'B'C + AB'C' + ABC = Σm(1,4,7)
```

- The **canonical SOP** is the OR of the minterms of every row where F = 1
- Read directly off a truth table with zero algebraic work
- Σm(…) lists the row numbers where the function is 1

Canonical product-of-sums (POS)

The same F is 0 for rows 0, 2, 3, 5 and 6

$F = M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6 = \Pi M(0,2,3,5,6)$

- The **canonical POS** is the AND of the maxterms of every row where F = **0**
- ΠM(…) lists the row numbers where the function is 0
- SOP lists where F = 1 and POS lists where F = 0: together they cover every row exactly once
- Both forms describe the exact same function

Standard forms

- Canonical forms are mechanical but rarely the simplest, because every term includes every variable
- A **standard SOP** is a sum of AND terms that need not include all the variables
- A **standard POS** is the equivalent for OR terms
- Standard forms are what you get after simplification (algebraic or K-map, Unit 2)
- Canonical forms are the unsimplified starting point

Worked example: canonical versus standard SOP


```
Canonical SOP:   F = A'B'C + AB'C' + AB'C
Standard SOP:    F = A'B'C + AB'
                 (AB'C' + AB'C = AB'(C' + C) = AB')
```

- Same function, fewer literals: from 9 literals to 5
- Two minterms that differ in only one variable were combined: the pattern K-maps are built to spot

1.7 Digital Logic Gates Overview

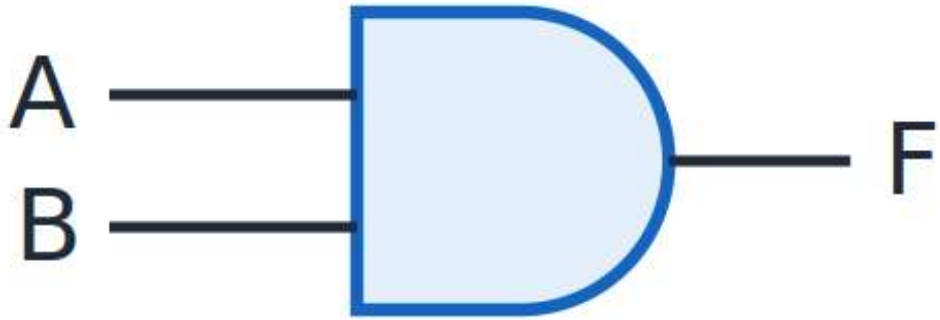
IN SIMPLE TERMS

A gate is a tiny circuit for one logic operation; NAND (or NOR) alone can build all the others.

The seven basic gates

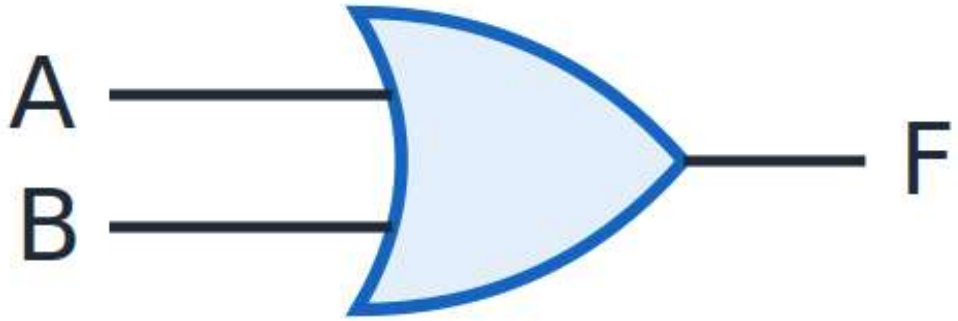
- A **logic gate** takes one or more binary inputs and produces one binary output, essentially instantly, based on the present inputs
- Standard shapes: a flat-backed, rounded-front “D” for **AND**, a curved pointed shield for **OR**, a triangle plus bubble for **NOT**
- A small **bubble** on a gate’s output means “and then invert the result”
- The next slides show each symbol, then its truth table

AND

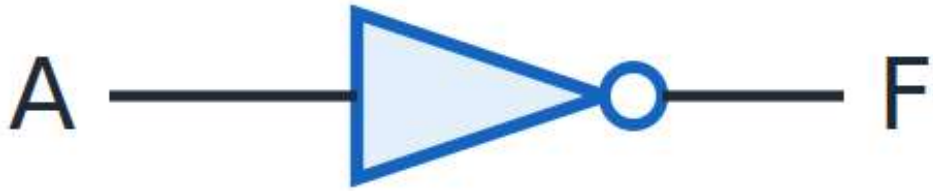


AND

OR

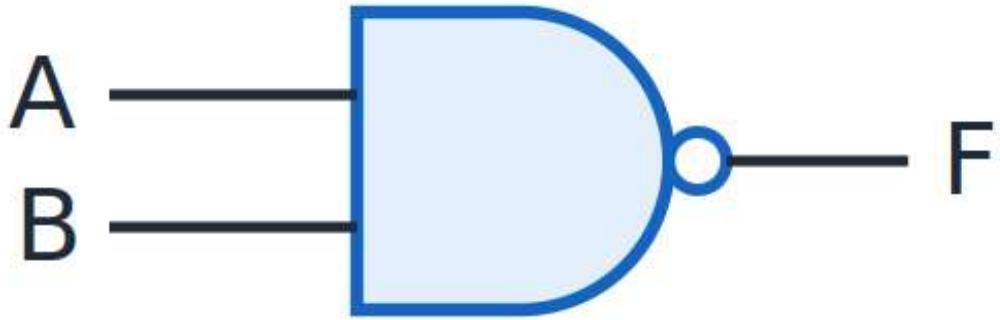


NOT



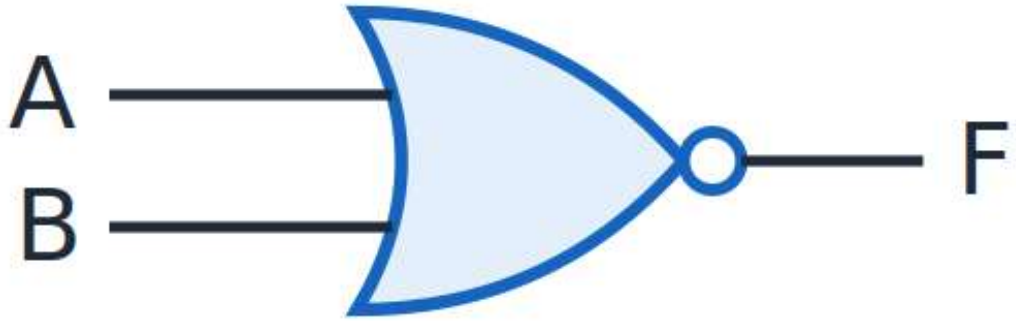
NOT

NAND



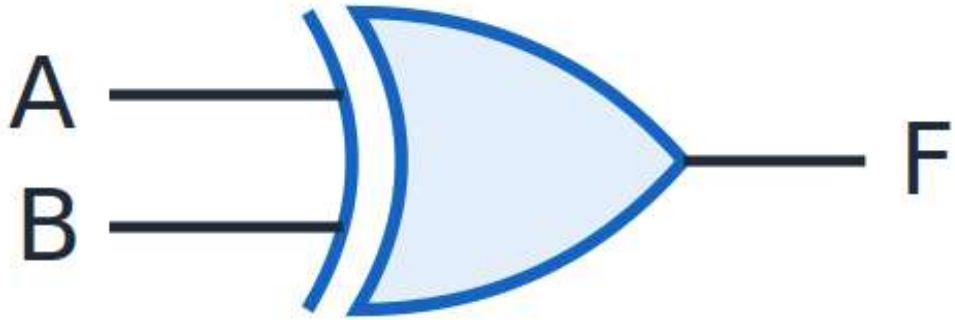
NAND

NOR

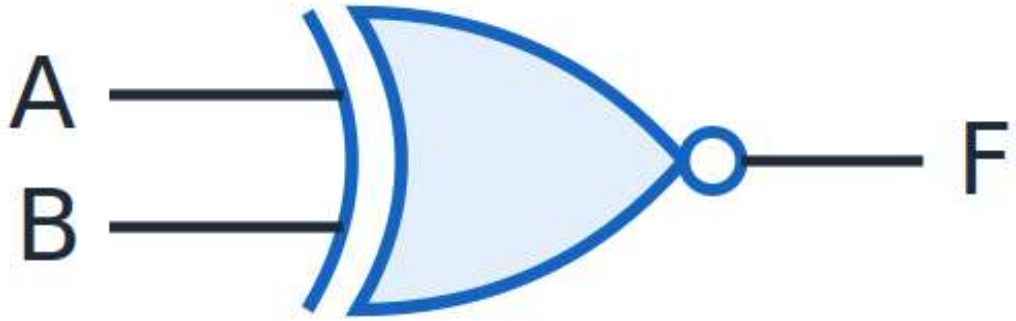


NOR

XOR



XNOR



XNOR

Truth tables of the two-input gates

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

Truth table of NOT

A	F
0	1
1	0

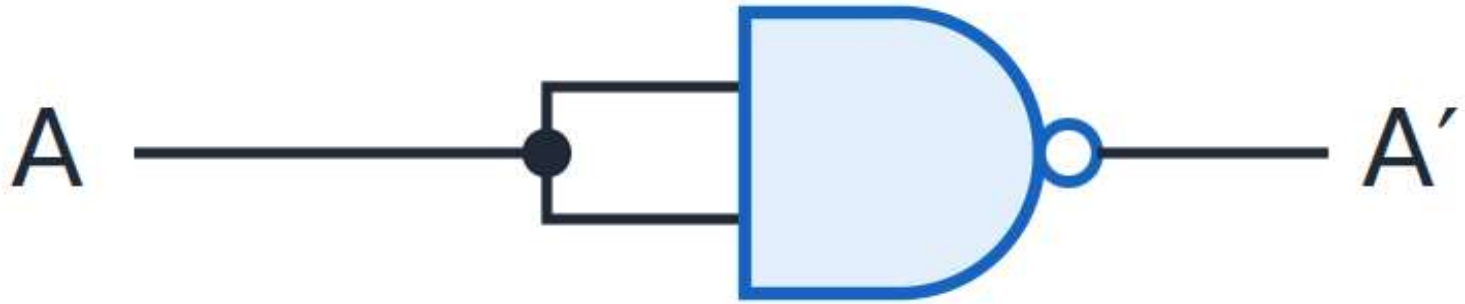
Derived gates

- AND, OR and NOT are the gates the postulates and theorems were written in terms of
- NAND, NOR, XOR and XNOR are all **derived** from those three
- NAND = AND then invert; XOR = “differs”, built from AND, OR and NOT
- They are common enough in real designs and chips to be treated as first-class gates with their own symbols

NAND and NOR are universal gates

- A gate (or set of gates) is **functionally complete** (“universal”) if any Boolean function can be built using only that gate, in any quantity
- **NAND alone** is universal, and so is **NOR alone**
- Every AND, OR and NOT gate you need can be built from nothing but NAND gates (or from nothing but NOR gates)
- The next slides build NOT, AND and OR from NAND only

NOT from NAND



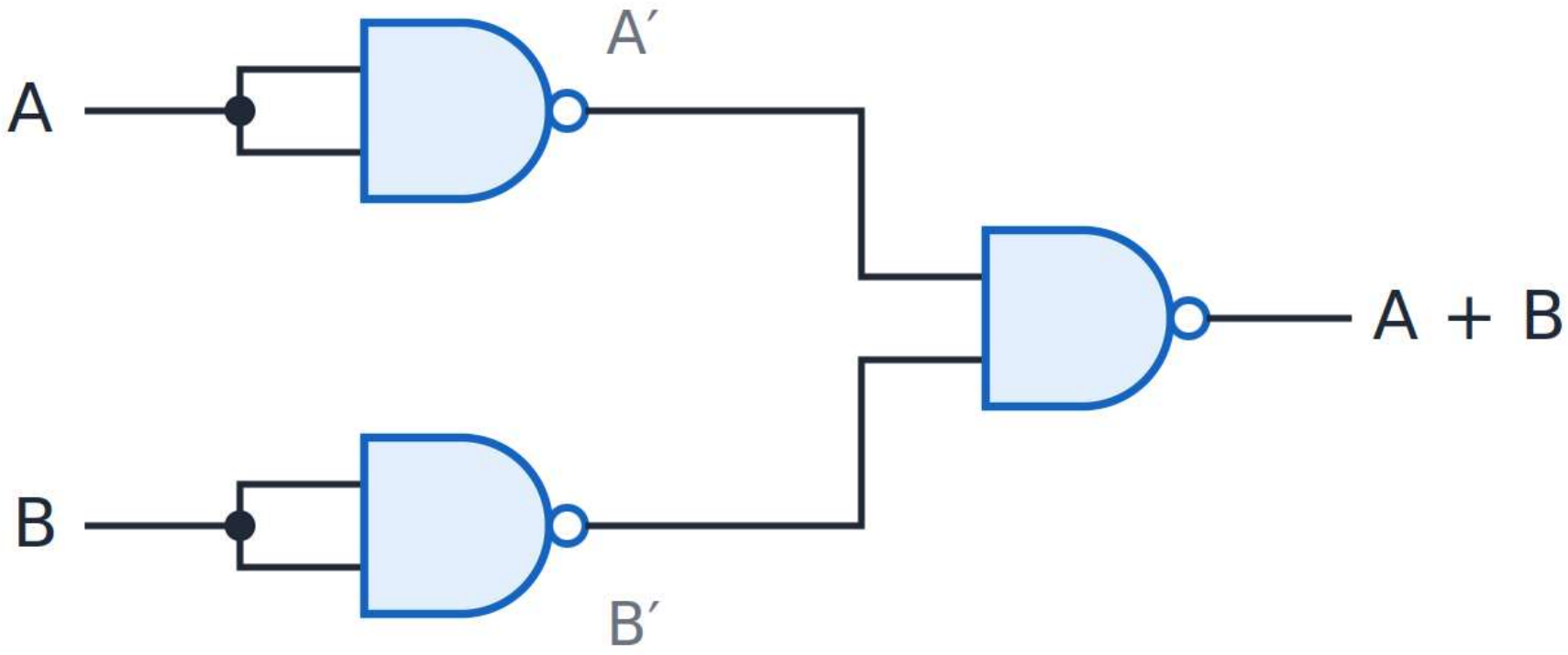
NOT from a single NAND with tied inputs.

AND from NAND



AND from two NANDs: the second NAND inverts the first.

OR from NAND



OR from three NANDs: invert A and B, then NAND them (DeMorgan).

Unit 1 summary

- **1.1 Number systems:** positional notation with radix r; repeated division and grouping for conversions
- **1.2 Fixed and floating point:** a fixed binary point, or a sign, mantissa and exponent (IEEE 754)
- **1.3 Complements:** 1’s complement flips bits; 2’s complement adds 1; subtract by adding, with the end-carry rule
- **1.4 Error detection and correction:** parity detects an error; Hamming code locates and corrects it
- **1.5 Boolean algebra:** postulates, duality, basic theorems and simplification
- **1.6 Canonical and standard forms:** minterms and maxterms, Σm and ΠM
- **1.7 Logic gates:** seven basic gates; NAND and NOR are universal

Review questions (1 of 2)

- **Q1.** Convert decimal 45 to binary, octal and hexadecimal
- **Q2.** Find the 1’s and 2’s complement of 1010 0110 and explain why 2’s complement has one zero
- **Q3.** Compute 6 − 9 using 4-bit 2’s complement and apply the end-carry rule
- **Q4.** Encode 5.625 in IEEE 754 single precision

Review questions (2 of 2)

- **Q5.** Encode data 1011 in Hamming(7,4) and locate a flipped bit using the syndrome
- **Q6.** Simplify F = AB + AB’ + A’B using the theorems
- **Q7.** Write F = Σm(1,4,7) as a canonical POS
- **Q8.** Build NOT, AND and OR from NAND gates only

References

- **Textbook:** M. Morris Mano and Michael D. Ciletti, Digital Design
- **Adapted figures:** none of the figures on these pages carries an “Adapted from” credit